

Absolutely. Here's a **complete Python set methods cheat sheet** with **syntax, properties, examples, and return values**.

Python Set — Methods & Properties

A **set** is an unordered, mutable collection of **unique and hashable elements**.

```
s = {10, 20, 30, 40}
```

1. Basic Properties

Property	Description	Example
Unordered	No fixed indexing/order	s = {1, 2, 3}
Unique	Duplicate values are automatically removed	{1, 2, 2, 3} → {1, 2, 3}
Mutable	Elements can be added/removed	s.add(5)
No indexing	Cannot use s[0]	✗ s[0]
No slicing	Cannot use s[1:3]	✗
Hashable elements only	List/dict/set cannot be elements	{[1,2]} ✗
Iterable	Can use for loop	for x in s:
No duplicate elements	Each element occurs once	{1,1,2} → {1,2}
Mutable set	set itself can change	s.add(10)
Immutable version	frozenset	frozenset({1,2})

2. Creating a Set

Using {}

```
s = {10, 20, 30}
print(s)
```

Using set()

```
s = set()
print(s)
```

⚠ Important:

```
x = {}  
print(type(x))
```

Output:

```
<class 'dict'>
```

An empty set must be:

```
x = set()
```

3. add()

Adds **one element** to the set.

Syntax

```
set.add(element)
```

Example

```
s = {1, 2, 3}
```

```
s.add(4)
```

```
print(s)
```

Possible output:

```
{1, 2, 3, 4}
```

Property	Value
Adds	One element
Modifies original set	Yes
Return value	None
Duplicate allowed?	No

```
s.add(3)
```

No change because 3 already exists.

4. update()

Adds **multiple elements** from another iterable.

Syntax

```
set.update(iterable)
```

Example

```
s = {1, 2, 3}
```

```
s.update([4, 5, 6])
```

```
print(s)
```

Output:

```
{1, 2, 3, 4, 5, 6}
```

You can pass multiple iterables:

```
s.update([4, 5], (6, 7), {8, 9})
```

Property	Value
Adds	Multiple elements
Accepts	List, tuple, set, string, etc.
Modifies original	Yes
Return	None

5. remove()

Removes a specified element.

```
s = {10, 20, 30}
```

```
s.remove(20)
```

```
print(s)
```

Output:

```
{10, 30}
```

Important

If the element doesn't exist:

```
s.remove(100)
```

You get:

KeyError

Property	Value
Removes	Specific element
If element exists	Removes it
If absent	KeyError
Return	None

6. discard()

Also removes a specific element.

```
s = {10, 20, 30}
```

```
s.discard(20)
```

The major difference:

```
s.discard(100)
```

No error.

remove() VS discard()

Method	Element exists	Element doesn't exist
remove()	Removes	✗ KeyError
discard()	Removes	✓ No error

7. pop()

Removes and **returns an arbitrary element**.

```
s = {10, 20, 30}
```

```
x = s.pop()
```

```
print(x)
```

```
print(s)
```

Because sets are unordered, you **should not depend on which element is removed**.

Property	Value
Removes	Arbitrary element
Returns	Removed element
Modifies set	Yes
Empty set	KeyError

8. clear()

Removes **all elements**.

```
s = {1, 2, 3, 4}
```

```
s.clear()
```

```
print(s)
```

Output:

```
set()
```

Property	Value
Removes	Everything
Original set modified	Yes
Return	None

9. copy()

Creates a **shallow copy** of the set.

```
s1 = {1, 2, 3}
```

```
s2 = s1.copy()
```

```
print(s2)
```

Now:

```
s2.add(4)
```

```
print(s1)
```

```
print(s2)
```

Output:

```
{1, 2, 3}
```

```
{1, 2, 3, 4}
```

10. union()

Combines elements from sets.

Syntax

```
A.union(B)
```

Example

```
A = {1, 2, 3}
```

```
B = {3, 4, 5}
```

```
C = A.union(B)
```

```
print(C)
```

Output:

```
{1, 2, 3, 4, 5}
```

Operator

A | B

Method	Operator
A.union(B)	A B

Does **not** modify the original sets.

11. intersection()

Returns elements common to both sets.

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5, 6}
```

```
print(A.intersection(B))
```

Output:

```
{3, 4}
```

Operator

A & B

Method	Operator
intersection()	&

12. difference()

Returns elements present in the first set but **not** in the second.

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5}
```

```
print(A.difference(B))
```

Output:

{1, 2}

Operator

A - B

Important:

A - B

is different from:

B - A

13. symmetric_difference()

Returns elements that are in **either set, but not both**.

A = {1, 2, 3}

B = {3, 4, 5}

```
print(A.symmetric_difference(B))
```

Output:

{1, 2, 4, 5}

Operator

A ^ B

Think:

Union – Intersection

14. intersection_update()

Updates the original set with only common elements.

A = {1, 2, 3, 4}

B = {3, 4, 5}

```
A.intersection_update(B)
```

```
print(A)
```

Output:

```
{3, 4}
```

Difference from `intersection()`

```
A.intersection(B)
```

creates a new set.

```
A.intersection_update(B)
```

changes A.

15. `difference_update()`

Removes elements from the original set that are present in another set.

```
A = {1, 2, 3, 4}
```

```
B = {3, 4, 5}
```

```
A.difference_update(B)
```

```
print(A)
```

Output:

```
{1, 2}
```

Equivalent idea:

```
A = A - B
```

but `difference_update()` modifies A in-place.

16. `symmetric_difference_update()`

Updates the original set with the symmetric difference.

```
A = {1, 2, 3}
B = {3, 4, 5}
```

```
A.symmetric_difference_update(B)
```

```
print(A)
```

Output:

```
{1, 2, 4, 5}
```

17. isdisjoint()

Checks whether two sets have **no common elements**.

```
A = {1, 2, 3}
B = {4, 5, 6}
```

```
print(A.isdisjoint(B))
```

Output:

```
True
```

Another example:

```
A = {1, 2, 3}
B = {3, 4, 5}
```

```
print(A.isdisjoint(B))
```

Output:

```
False
```

18. issubset()

Checks whether **all elements of one set exist in another set**.

```
A = {1, 2}
B = {1, 2, 3, 4}
```

```
print(A.issubset(B))
```

Output:

True

Operator

A <= B

19. issuperset()

Checks whether a set contains **all elements of another set**.

A = {1, 2, 3, 4}

B = {1, 2}

```
print(A.issuperset(B))
```

Output:

True

Operator

A >= B

Complete Set Method Table

Method	Purpose	Modifies Set?	Returns
add()	Add one element	✓	None
update()	Add multiple elements	✓	None
remove()	Remove element	✓	None
discard()	Remove element safely	✓	None
pop()	Remove arbitrary	✓	Element

Method	Purpose	Modifies Set?	Returns
	element		
clear()	Remove all elements	✓	None
copy()	Create shallow copy	✗	New set
union()	Combine sets	✗	New set
intersection()	Common elements	✗	New set
difference()	Elements only in first set	✗	New set
symmetric_difference()	Elements in either, not both	✗	New set
intersection_update()	Keep common elements	✓	None
difference_update()	Remove common elements	✓	None
symmetric_difference_update()	Keep non-common elements	✓	None
isdisjoint()	Check no common elements	✗	True/False
issubset()	Check subset	✗	True/False
issuperset()	Check superset	✗	True/False

Set Operations Cheat Sheet

Suppose:

A = {1, 2, 3}

B = {3, 4, 5}

Operation	Method	Operator	Result
Union	A.union(B)	$A \cup B$	{1,2,3,4,5}
Intersection	A.intersection(B)	$A \cap B$	{3}
Difference	A.difference(B)	$A - B$	{1,2}
Reverse Difference	B.difference(A)	$B - A$	{4,5}
Symmetric Difference	A.symmetric_difference(B)	$A \oplus B$	{1,2,4,5}
Subset	A.issubset(B)	$A \subseteq B$	False
Superset	A.issuperset(B)	$A \supseteq B$	False
Disjoint	A.isdisjoint(B)	—	False

One Important Concept: update VS union

This is a common interview question.

union()

A = {1, 2}

B = {2, 3}

C = A.union(B)

print(A) # {1, 2}

print(C) # {1, 2, 3}

Original A remains unchanged.

update()

A = {1, 2}

B = {2, 3}

A.update(B)

print(A) # {1, 2, 3}

Original A changes.

Remember:

union() → creates a new set
update() → modifies the existing set

Set vs Frozenset

Feature	set	frozenset
Mutable	✓	✗
add()	✓	✗
remove()	✓	✗
update()	✓	✗
union()	✓	✓
intersection()	✓	✓
Can be dictionary key	✗	✓
Hashable	✗	✓

Example:

```
fs = frozenset([1, 2, 3])
```

```
print(fs)
```

You cannot do:

```
fs.add(4)
```

because frozenset is immutable.

Quick memory trick

Modify methods:

add → update → remove → discard → pop → clear

Set-operation methods:

union → intersection → difference → symmetric_difference

In-place operation methods:

intersection_update → difference_update → symmetric_difference_update

Relationship/check methods:

issubset → issuperset → isdisjoint

Copy:

copy()